

Spark(Exercise)

Friday, August 01, 2025 10:59 AM

Beginner Level

Conceptual Questions

1. What is Apache Spark, and why is it faster than Hadoop MapReduce?
2. Name two common use cases for Spark in real-world applications.
3. What are the main components of the Spark ecosystem?
4. What is the difference between RDDs and DataFrames?
5. Explain the concept of lazy evaluation in Spark.
6. What triggers Spark to actually execute transformations?
7. In Spark architecture, what is the role of the Driver and Executors?
8. What is Spark SQL, and why would you use it?

Hands-On Exercises

1. Create a SparkSession in PySpark and load a CSV file named employees.csv into a DataFrame. Print schema and first 5 rows.
2. Count the total number of employees from employees.csv.
3. Select only the name and department columns from the employees DataFrame.
4. Filter employees with salary greater than 80000 and display them.
5. Sort employees by salary in descending order.
6. Create a temporary view from the employees DataFrame and run a SQL query to get the average salary per department.

Intermediate Level

Conceptual Questions

1. Explain the difference between narrow and wide transformations with examples.
2. What is a Spark DAG, and how does it optimize execution?
3. Compare Spark deployment modes: Local, YARN, Standalone, and Kubernetes.
4. When would you use broadcast joins in Spark?
5. What are the advantages of using Parquet or ORC over CSV in Spark?
6. What is the Catalyst Optimizer, and why is it important?
7. Explain the difference between repartition() and coalesce().
8. Why is Spark fault-tolerant?

Hands-On Exercises

1. Create a DataFrame from students.csv and perform a groupBy on name's first letter (hint: use substring). Count students in each group.
2. Join employees.csv with departments.csv using dept_id = id. Show employee names with their department names.
3. Perform an aggregation to find the maximum salary in each department.
4. Load sales_2024-01-01.csv and sales_2024-01-02.csv, union them into a single DataFrame, and compute total sales.
5. Drop duplicate employee names from employees.csv and show results.
6. Handle nulls by replacing missing salary values with 0 in employees.csv.
7. Write employees DataFrame into Parquet format with Snappy compression.

8. Cache the employees DataFrame and check if it is cached using Spark catalog.

Advanced Level

Conceptual Questions

1. How does Spark handle stage division in jobs, and how do wide transformations affect this?
2. Explain the differences between RDD, DataFrame, and Dataset APIs.
3. What is vectorized query execution in Spark, and when is it beneficial?
4. Discuss strategies to deal with data skew in Spark joins.
5. How do Spark Window functions differ from groupBy aggregations?
6. What role do Tungsten and Catalyst play in Spark performance tuning?
7. When would you choose Spark Structured Streaming over Spark Streaming?
8. How do you tune partition sizes in Spark for optimal performance?

Hands-On Exercises

1. Load transactions_large.csv into a DataFrame. Partition the data by transaction_date and write it into Parquet files.
2. Use a Window function to rank employees by salary within each department.
3. Implement a rolling average of revenue in a DataFrame using a Window specification.
4. Perform a broadcast join between a large sales DataFrame and a small departments DataFrame.
5. Benchmark query performance by running the same aggregation on CSV, Parquet, and ORC formats.
6. Simulate skewed data by creating many rows with the same join key and resolve using salting.
7. Repartition the transactions DataFrame into 8 partitions and measure execution time for an aggregation. Then coalesce to 4 and compare.
8. Create a Spark SQL query to compute top 3 customers by total transaction amount using a temp view.



spark_pract...

